University of Inland Norway

Faculty of Audiovisual Media and Creative Technologies

Bachelor thesis

Saving the Game: Consistent Offline-Cloud State Synchronization in Survival Video Games

Lagre spillet:
Konsistent synkronisering av lokal- og skybasert tilstand i overlevelsesspill

B2VISIM Bachelor in Game Technology and Simulation

SPIS2900 Bachelor thesis

Hans Alan Whitburn Haugen

April 27, 2026

Abstract

Many survival games store their world entirely on remote servers. This means players cannot progress offline, and if the developer shuts down the online service, the game is lost forever. This thesis investigates ways to design a system that supports both offline and online gameplay, and that allows a player to transition from an offline session to an online one without losing progress.

The main point is to log each player action in the form of a command instead of making the changes live on the server immediately. When the player is offline, the commands accumulate in a local queue. As soon as the connection is re-established, these commands are executed on the authoritative server in the same order. The biggest issue is how to reconcile conflicts: for example, if during the period of being offline, the building that a player intended to demolish was destroyed by another player, or the same territory was grabbed by another player, etc. then the system needs to figure out how to handle such situations. This thesis proposes a solution that divides commands into categories: there are commands that can always be executed irrespective of changes, some need a version check, and some still have to be rejected and communicated to the player.

A prototype was built in C++ to validate the design, including a SQLite-backed offline database and a durable command log that survives crashes.

Sammendrag

Mange overlevelsesspill lagrer spillverdenen på eksterne servere. Dette gjør det umulig å spille spillet offline, og hvis utvikleren går konkurs mister spillerne tilgang til spillet. Denne oppgaven undersøker hvordan et system kan utvikles som gjør at en kan spille spillet uten internett, og så synkronisere spillverdenen ved å koble seg til en server når en spiller selv ønsker det.

Denne bacheloren foreslår en metode som går ut på å loggføre hver spillerhandling som kommandoer (Command-pattern), i stedet for å sende ny tilstand direkte til serveren. Mens spilleren spiller frakoblet, samles disse kommandoene opp lokalt i en kø (queue datastruktur). Når forbindelsen er gjenopprettet, spilles de av mot den autoritative serveren i riktig rekkefølge. Det vanskelige er å avgjøre hva som skal skje når ved konflikter: for eksempel, hva skjer når annen spiller har revet ned den samme bygningen eller inntatt det samme territoriet mens en spilte frakoblet? Denne bacherloroppgaven foreslår konflikthåndteringsstrategier som klassifiserer kommandoer etter type: det er kommandoer som alltid utføres uavhengig av hva som har endret seg, andre krever en versjonssjekk, og noen må rett og slett avvises og rapporteres tilbake til spilleren.

En prototype ble bygget i C++, inkludert en SQLite-basert frakoblet database og en kommandoliste som overlever at spillet plutselig lukkes.

Contents

1 Introduction **1**

2 Methodology **2**

3 Related Work **3**
3.1 Game Industry Approaches 3
3.2 Distributed Systems Approaches 3

4 Typical Challenges in Persistence Systems **5**
4.1 Limitations of Server-Authoritative Architectures 6
4.2 Dependence on Continuous Connectivity 6
4.3 Absence of Local Persistence Authority 6
4.4 Lack of Divergence Handling Mechanisms 6
4.5 Scalability vs. Flexibility Trade-off 7

5 Proposed Hybrid Persistence Architecture **8**
5.1 Command-Based State Replication 8
5.2 Implementation considerations 9
5.3 Conflict Resolution Strategy 9
5.3.1 Conflict Detection via Entity Versioning 10
5.3.2 Command Classification 10
5.3.3 Resolution Policies 10
5.3.4 Rejection Manifest 11
5.4 Integration Possibilities 11
5.5 Local Persistence via SQLite 12
5.6 ORM (Object-Relational Mapping) 13
5.7 Prototype Implementation 13

6 Evaluation **15**
6.1 Correctness 15
6.2 Replay Throughput 15
6.3 Command Log Backend Comparison 16
6.4 Conflict Detection Overhead 16
6.5 Full Offline Session Round-Trip 16
6.6 Command Log Disk Footprint 17

7 Discussion **18**
7.1 Architectural Trade-offs 18
7.2 Evaluation Findings 18
7.3 Scalability Considerations 19
7.4 Limitations and Threats to Validity 19
7.5 Game Preservation 20

8 Future Work **21**

9 Conclusion **22**

Listings

1 DBI interface (abstract base class) 5
2 Command structure and local command log 8
3 Server-side command validation and conflict resolution 10
4 Connectivity-aware DBI factory 11

1 Introduction

This thesis considers ways to develop the persistence system of a modern large-scale survival multiplayer game. The designs target persistent multiplayer worlds shared by thousands of players, and are intended to support the storage of player progression, world state, and faction dynamics. They are not concerned with ephemeral information such as player position or real-time combat replication; the focus is on long-term persistent data of the kind typically stored in centralized, cloud-hosted SQL databases.

The suggested designs are intended to provide ways to support both offline and online functionality. Having the ability to make a game run offline is important, as on console platforms access to online multiplayer features typically requires a paid subscription service. A significant portion of console players therefore do not participate in online multiplayer environments (Baker, 2025). Playing with friends should be supported via local cooperative modes while preserving core gameplay mechanics.

This requirement introduces architectural challenges which need to be addressed sensibly. Games on the market which support persistent game state stored exclusively in cloud-based SQL databases assume continuous server connectivity. Supporting offline gameplay necessitates a hybrid architecture in which persistent world state can be stored locally, potentially using embedded database solutions such as SQLite which can later be synchronized with the authoritative cloud-based server infrastructure.

Many gamers prefer to play offline, but might want to connect online as they advance in the persistent world. Collaboration with other players is often necessary in survival games to navigate the harsh environment. Political dynamics and faction-based conflict play a central role in survival games, as groups of players compete for control and influence within the game's persistent world. Designs that allow users to transition online after initial offline sessions are explored.

This thesis investigates the following research statement: How can synchronization algorithms for persistent game state be designed and evaluated with respect to consistency, conflict resolution, and scalability in large-scale survival multiplayer games?

2 Methodology

This thesis follows a design science research approach (Hevner et al., 2004), in which the primary output is an artifact — in this case an architectural design and a working software prototype — rather than an empirical study. The research proceeds in three stages.

The first stage is analysis: examining how existing server-authoritative persistence systems work and identifying where they break down when offline play is required. This is covered in Section 4.

The second stage is design: proposing a hybrid architecture based on the Command Pattern, specifying conflict resolution policies, and detailing how the design integrates with both online SQL backends and a local SQLite store. The design decisions are documented in Section 5.

The third stage is evaluation: implementing the design as a C++ prototype and assessing it against two criteria. First, correctness — every component is covered by automated unit and integration tests written with the Catch2 framework. Second, performance — a benchmark suite measures replay throughput, command log scalability, conflict detection overhead, and the end-to-end cost of a full offline session. The benchmarks also exposed and drove the fix of a quadratic performance issue in the initial implementation. Results are presented in Section 6.

The prototype is intentionally self-contained and does not require a running SQL server. This keeps the evaluation reproducible and focused on the algorithmic properties of the design rather than network or infrastructure variability.

3 Related Work

3.1 Game Industry Approaches

Several commercially released games have addressed aspects of offline-online synchronization, though typically without exposing the underlying mechanisms.

Valheim (Studio, 2021) stores its world state entirely in local files on the hosting player's machine. This makes the game fully playable offline and without any cloud dependency, but it provides no mechanism for merging divergent state: if two players host separate copies of the same world while disconnected, there is no supported way to reconcile the differences. This is effectively the snapshot problem described in Section 4 — straightforward for single-player scenarios but unworkable for shared persistent worlds.

Steam Cloud Save (Corporation, 2023) offers file-level synchronization across devices, uploading and downloading save files when a player connects. Conflicts are resolved by taking the most recently modified file, which is a form of last-write-wins and loses any changes made on the other device. This approach is simple and adequate for single-player games, but cannot handle concurrent modifications from multiple players.

Many large-scale survival multiplayer games — such as Rust and DayZ — take the opposite approach and make offline play impossible, requiring a persistent connection to a central server at all times. This sidesteps synchronization entirely but at the cost of accessibility.

3.2 Distributed Systems Approaches

The academic distributed systems literature offers more principled solutions, though they come with trade-offs in implementation complexity.

Conflict-free Replicated Data Types (CRDTs) (Shapiro et al., 2011) are mathematical data structures that guarantee eventual consistency without coordination. Any two replicas can be merged in any order and will converge to the same result. CRDTs would eliminate the need for conflict resolution policies entirely, but they require redesigning every game data type to conform to CRDT semantics — a significant architectural investment. They are also better suited to data types like counters and sets than to the complex relational structures (buildings with positions, quest state, faction graphs) that survival games require.

Operational Transformation (OT) (Ellis & Gibbs, 1989), the technique underlying collaborative editing tools such as Google Docs, transforms concurrent operations so that they can be applied in any order while producing a consistent result. The intent-based approach used in this thesis shares similarities with OT: both record operations rather than snapshots. However, OT requires a central server to sequence and transform operations in real time, which is not available during an offline session.

Event sourcing (Fowler, 2005) is a software architecture pattern in which state is derived entirely from an append-only log of events. The command log proposed in this thesis is closely related: both record intent rather than state, and both allow the current state to be reconstructed by replaying the log. The key difference is that event sourcing typically assumes a single authoritative log, whereas this thesis addresses what happens when two divergent logs (client offline, server online) must be merged.

The conflict resolution strategy — particularly entity versioning and optimistic concurrency control — follows the approach described by Bernstein and Goodman (Bernstein & Goodman, 1981), where transactions validate preconditions at commit time rather than acquiring locks upfront.

4 Typical Challenges in Persistence Systems

Game worlds consist of actors that may own inventory items, buildings, vehicles, navigation markers, abilities, and quest progression. In large-scale online survival games, persistent game state is commonly stored in centralized persistence systems backed by SQL databases hosted on cloud infrastructure.

Such systems typically follow a server-authoritative architecture in which all state-modifying actions are processed through a centralized backend. Player actions are buffered temporarily on the client before being committed to the authoritative server. Multiple server instances may run concurrently, but at any given time only one instance holds authoritative control over a given player or region. Actions are executed as database transactions, providing rollback in the event of failure.

A common design pattern for this kind of system is to apply the Service Locator pattern to abstract the communication layer responsible for generating and dispatching persistence commands. This abstraction is hereafter referred to as the Database Interface (DBI). Rather than issuing raw SQL queries directly from game code, the DBI invokes predefined server-side functions. Each function executes as a transactional unit and can be rolled back if any step fails, so every state-modifying action is encapsulated as an atomic database operation.

A well-designed DBI supports atomicity, consistency, and idempotency. If a network failure occurs, or if the client is uncertain whether a transaction completed, the DBI can safely retry without risking duplicate or inconsistent state changes. An important structural advantage of this abstraction is that the DBI can be swapped out via the Service Locator pattern: an online session routes calls to the remote SQL backend, while an offline session could route them to a local implementation — the implications of which are explored in Section 5.

A SQL-based DBI architecture treats the cloud-hosted SQL database as the single authoritative source of truth for all persistent game state. This provides strong consistency guarantees in fully online environments, but it inherently depends on centralized authority and does not account for state divergence in disconnected scenarios.

Listing 1: DBI interface (abstract base class)

```
class IDatabaseInterface
{
public:
    virtual ~IDatabaseInterface() = default;

    virtual bool CreateBuilding(int32_t playerId, int32_t
        buildingTypeId,
                                float posX, float posY,
                                    float posZ) = 0;
    virtual bool DestroyBuilding(int32_t buildingId) = 0;
```

```
    virtual bool TransferItem(int32_t fromPlayerId, int32_t
        toPlayerId,
                              int32_t itemId, int32_t
                                  quantity) = 0;

    virtual bool CompleteQuest(int32_t playerId, int32_t
        questId) = 0;

    virtual bool UpdateFactionRelation(int32_t factionA,
        int32_t factionB,
                                       int32_t relationDelta
                                           ) = 0;
};
```

4.1 Limitations of Server-Authoritative Architectures

A server-authoritative persistence architecture operates under the assumption of continuous connectivity to a centralized cloud infrastructure. While this model ensures strong consistency and transactional integrity in online multiplayer environments, it introduces several limitations when extended to offline or hybrid scenarios.

4.2 Dependence on Continuous Connectivity

Because all state-modifying actions are executed as remote transactions, gameplay cannot progress without server access. This dependency prevents offline play entirely and limits deployment flexibility on platforms where online access may be restricted or requires a paid subscription service.

4.3 Absence of Local Persistence Authority

In a purely server-authoritative model, the client does not maintain an authoritative local representation of persistent world state. Although temporary state may exist in memory during a session, durable storage is exclusively managed by the remote infrastructure. This prevents local world progression in disconnected scenarios and complicates support for local cooperative play modes.

4.4 Lack of Divergence Handling Mechanisms

A server-authoritative architecture assumes a single linear progression of state changes and does not account for concurrent or divergent modifications occurring in separate environments — for example, an offline client and the live server both modifying the same entity. Consequently, such architectures typically lack:

1. Version tracking mechanisms
2. Conflict detection strategies

3. Deterministic merge policies

Without these components, safely synchronizing locally modified state with cloud-based state is not possible.

4.5 Scalability vs. Flexibility Trade-off

A centralized server-authoritative design scales well for persistent multiplayer environments but sacrifices flexibility. Extending it toward hybrid offline–online play requires introducing distributed state management — concerns that a purely server-authoritative architecture was not originally designed to address.

5 Proposed Hybrid Persistence Architecture

5.1 Command-Based State Replication

To enable offline progression while preserving compatibility with a server-authoritative model, this thesis proposes a command-based persistence mechanism inspired by the Command Pattern as described in Design Patterns: Elements of Reusable Object-Oriented Software and further discussed in Game Programming Patterns (Nystrom, 2014).

Rather than directly modifying persistent state in the cloud, player actions are encapsulated as serializable command objects. Each command represents an intention to modify game state (e.g., create building, transfer item, complete quest) along with the necessary parameters to execute the modification.

During offline gameplay, commands are appended to a durable local command log instead of being transmitted immediately to the server. When connectivity is restored, the queued commands are transmitted sequentially to the authoritative backend infrastructure and executed as transactional operations.

This approach introduces several advantages:

1. Commands are naturally serializable and can be stored locally.
2. Replay of commands enables deterministic reconstruction of state.
3. The command log provides a history of offline actions.
4. Existing server-side transactional guarantees can remain unchanged.

By storing intent rather than derived state, the system avoids directly merging divergent world snapshots and instead synchronizes through ordered execution of state-modifying operations.

Listing 2: Command structure and local command log

```
struct Command
{
    std::string type;            // e.g. "CreateBuilding", "
        ResourceDelta"
    int64_t     timestamp;       // UTC milliseconds at time
         of action
    std::string idempotencyKey;  // UUID to detect duplicate
         replays
    std::string entityId;        // ID of the affected
        entity
    int32_t     expectedVersion; // entity version at
        command creation
    int32_t     delta;           // signed delta for
        commutative operations
};
```

```
class CommandLog
{
public:
    void                    Append(const Command& cmd);
    std::vector<Command>    GetPending() const;
    void                    MarkFlushed(const std::string&
        idempotencyKey);
    void                    MarkFlushedBatch(
                                const std::vector<std::string>&
                                    keys);
    std::size_t             PendingCount() const;

private:
    std::vector<Command>    m_pending;
};
```

5.2 Implementation considerations

This system must be designed with integrity guarantees in mind. Since commands are generated client-side and stored locally, a malicious client could potentially forge or alter commands before transmission. Every action must therefore be validated on the server before it is committed, and any command that produces an invalid or impossible state transition must be rejected. Conflict detection, entity versioning, and per-type resolution policies are addressed in the following subsection.

Beyond conflict handling, two additional concerns must be managed. First, ordering guarantees: the command log must preserve the sequence in which actions were taken offline, and the server must replay them in that order to ensure deterministic results. Second, idempotency and duplicate detection: if a replay attempt fails partway through or a command is transmitted more than once, the server must be able to identify and skip already-applied commands. The idempotency key stored on each command serves this purpose.

A remaining open concern is command dependency on server-side state that changed during the offline session. If an offline command assumed the existence of an entity that was deleted by another player while the client was disconnected, the command cannot be applied even if its version check would otherwise pass. Handling such cases requires pre-condition checks on the server that go beyond version comparison, and the rejection manifest provides the mechanism for communicating these failures back to the client.

5.3 Conflict Resolution Strategy

When offline commands are replayed against the authoritative server, conflicts arise when the server state has diverged from the state against which those commands were originally generated. A command that was valid at the time of

creation may produce an invalid or inconsistent state transition when applied to an evolved world.

5.3.1 Conflict Detection via Entity Versioning

Each persistent entity (e.g. a building, an inventory slot, or a faction) is assigned a monotonically increasing version number that is incremented with every committed state change. When a command is generated offline, the version of each affected entity is recorded alongside the command parameters. Upon replay, the server compares the recorded version against the current entity version. A mismatch indicates that the entity was modified after the command was issued, signalling a potential conflict.

This mechanism mirrors optimistic concurrency control (OCC) in traditional database systems, where transactions proceed without locking but are validated at commit time (Bernstein & Goodman, 1981).

5.3.2 Command Classification

Not all version mismatches require rejection. Commands can be classified by their conflict behaviour:

1. **Idempotent commands** — Quest completion, achievement unlocks, and similar one-time flags. These produce the same result regardless of how many times they are applied and can be safely replayed unconditionally.
2. **Commutative commands** — Additive operations such as resource collection or faction reputation deltas. Because the order of application does not affect the final value, these can be merged without conflict by summing the deltas.
3. **Non-commutative commands** — Destructive or positional operations such as demolishing a building or placing a structure on a specific grid cell. These depend on preconditions that may have been invalidated by independent server-side changes.

5.3.3 Resolution Policies

For each conflict class, a deterministic resolution policy is applied server-side before any command is committed.

Listing 3: Server-side command validation and conflict resolution

```
ReplayResult ReplayCommand(const Command& cmd, Database& db,
                           RejectionManifest& manifest)
{
    // Idempotent: already applied, skip silently
    if (db.IsAlreadyApplied(cmd.idempotencyKey))
        return ReplayResult::Duplicate;
```

```
    // Commutative: apply delta regardless of entity version
    if (cmd.type == "ResourceDelta" || cmd.type == "
        ReputationDelta")
        return db.ApplyDelta(cmd);

    // Non-commutative: validate entity state before
        committing
    Entity entity = db.GetEntity(cmd.entityId);

    if (!entity.exists)
    {
        manifest.push_back({cmd, ReplayResult::Rejected});
        return ReplayResult::Rejected;
    }

    if (entity.version != cmd.expectedVersion)
    {
        manifest.push_back({cmd, ReplayResult::Conflict});
        return ReplayResult::Conflict;
    }

    return db.Commit(cmd);
}
```

5.3.4 Rejection Manifest

Commands that fail validation are not silently discarded. Instead, they are collected into a rejection manifest that is returned to the client after the replay pass completes. The manifest records the command type, the reason for rejection, and the current server-side state of the affected entity. This allows the game to inform the player of which offline actions could not be applied and why, enabling the client to present meaningful feedback rather than silently losing progress.

5.4 Integration Possibilities

Currently, every action that should persist goes through the DBI. By extending the DBI into a factory that produces command objects, the system can be made to route actions in two ways depending on connectivity state: online commands are queued and transmitted to the server immediately as before, while offline commands are serialized and stored on disk, deferred until the player reconnects.

The routing decision is straightforward to implement. A factory function checks connectivity and returns the appropriate DBI implementation:

Listing 4: Connectivity-aware DBI factory

```
std::unique_ptr<IDatabaseInterface> CreateDBI(bool isOnline)
{
    if (isOnline)
```

```
        return std::make_unique<RemoteSQLDBI>(serverConfig);
    else
        return std::make_unique<SQLiteDBI>("offline_state.db
            ");
}
```

Game code calls DBI methods without knowing which backend is active. When connectivity is restored, the engine drains the durable command log by calling `ReplayAll`, which submits each queued command to the server in order and returns a `RejectionManifest` listing any commands that could not be applied.

Figure 1 shows the overall data flow.

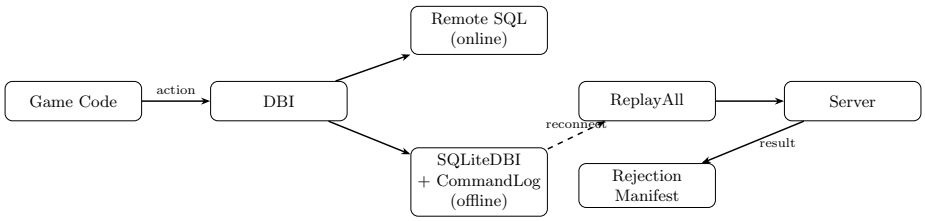

Figure 1: Hybrid persistence architecture: game code interacts only with the DBI; the active implementation is swapped at runtime depending on connectivity.

5.5 Local Persistence via SQLite

A direct approach to enabling offline functionality is to extend the Database Interface (DBI) abstraction with a complete SQLite-backed implementation. Rather than communicating with remote SQL servers, the DBI would execute equivalent operations against a locally embedded SQLite database during offline sessions.

SQLite is an embedded, serverless, ACID-compliant relational database engine that operates as a lightweight C/C++ library. Unlike other SQL implementations, it does not require a separate server process; instead, it runs within the host application and stores data in a single file on disk. This makes it well-suited for offline persistence in desktop and console environments.

By preserving the DBI abstraction layer, the system can maintain a consistent programming interface for persistence operations regardless of connectivity state. When offline, DBI calls would be routed to the SQLite backend. When online, calls would be directed to the existing SQL infrastructure.

This design minimizes disruption to the gameplay code, as the persistence boundary remains unchanged. However, it introduces challenges related to schema compatibility, transaction semantics, and synchronization between the local SQLite instance and the authoritative cloud database.

5.6 ORM (Object-Relational Mapping)

An alternative architectural approach considered was the introduction of an Object–Relational Mapping (ORM) system. An ORM provides a layer that maps in-memory C++ objects directly to relational database structures, allowing persistent entities to be stored and retrieved without writing explicit SQL logic. In such a system, game objects (e.g., inventory items, buildings, or quests) would be automatically translated into database rows and reconstructed from relational data.

ORM frameworks are common in higher-level languages such as Python and Java, where reflection and dynamic typing simplify runtime type inspection and schema generation. In contrast, implementing a fully featured ORM in C++ presents additional complexity due to the lack of built-in reflection and the need for explicit type registration and serialization logic.

While an ORM could reduce manual database interaction and potentially simplify persistence logic, several factors limited its suitability for this project. First, the DBI abstraction connected to predefined SQL functions is already a stable persistence boundary. Introducing an ORM would require significant restructuring of that boundary. Second, the project required explicit support for SQLite in offline mode, meaning that any ORM solution would need to maintain compatibility across multiple database backends. Finally, time constraints and integration risks made a full ORM implementation impractical within the scope of this bachelor project.

For these reasons, the project retained the existing DBI abstraction and instead focused on extending it to support hybrid online–offline persistence rather than replacing it with a generalized object–relational mapping layer.

5.7 Prototype Implementation

A self-contained C++ prototype of the proposed architecture was developed alongside this thesis and is available at:

https://github.com/alanhaugen/SPIS2900-Bachelor

The prototype is located in the `app/` directory and is built with CMake. It depends only on Catch2, which is fetched automatically at configure time, and a system SQLite3 library. The implementation is organized as two libraries:

`offline_sync_lib` contains the core persistence abstractions:

- `IDatabaseInterface` — the abstract DBI used by game code, with virtual methods for all persistent game actions.
- `Command` and `CommandLog` — the in-memory command queue described in Section 5.
- `Database` — a lightweight in-memory entity store used for testing, with version tracking and idempotency key recording.

- `ReplayCommand` / `ReplayAll` — the conflict-resolution algorithm from Section 5, including the `RejectionManifest` type.

`offline_sync_sqlite_lib` contains the SQLite-backed implementations:

- `SQLiteDBI` — a concrete implementation of `IDatabaseInterface` that stores game state in a local SQLite file. It maintains game-domain tables for buildings, item inventories, completed quests, and faction relations. Item transfers execute inside an explicit SQLite transaction and are rejected atomically if the sender has insufficient inventory. Quest completion uses `INSERT OR IGNORE` to preserve idempotency. Faction relation updates use an UPSERT to accumulate signed deltas.
- `SQLiteCommandLog` — a durable replacement for the in-memory `CommandLog` that writes each command to a SQLite file before returning. The `idempotency_key` column carries a `UNIQUE` constraint so that a double-append is silently discarded at the database level. Commands survive process restarts and can be recovered on next open.

The prototype includes 31 Catch2 unit tests covering the command log, idempotency, commutative merging, non-commutative conflict detection, SQLite persistence, and a full offline-session integration scenario.

6 Evaluation

The prototype was evaluated along two dimensions: correctness, verified through automated testing, and performance, measured through a benchmark suite run on the same hardware used for development (Apple M-series, macOS, Release build with compiler optimisations enabled).

6.1 Correctness

The implementation is covered by 31 Catch2 unit and integration tests organised into five categories: command log behaviour (append, ordering, flush), idempotency (duplicate commands produce no side effects), commutative merging (resource and reputation deltas apply regardless of version mismatch), non-commutative conflict detection (version mismatch and missing entities are caught and reported), and a full offline session integration test that exercises all result types in a single replay pass. All 31 tests pass.

6.2 Replay Throughput

The first benchmark measures how quickly the in-memory replay engine can process a command log of increasing size, using one unique entity per command to isolate the cost of the replay loop itself from conflict resolution overhead.

Commands	Time (ms)	Throughput (cmd/s)	Applied
100	0.02	4,907,975	72
500	0.09	5,780,347	368
1,000	0.16	6,448,160	734
5,000	0.81	6,196,747	3,752
10,000	1.44	6,957,533	7,418
50,000	9.36	5,340,668	37,018

Throughput is consistent at approximately 5–7 million commands per second across all log sizes, confirming that replay scales linearly with the number of commands. A log of 10,000 commands — large by the standards of a single offline session — replays in under 2 milliseconds, well below any perceptible threshold. The applied count is lower than the total because 30% of commands are commutative and increment the entity version, causing subsequent non-commutative commands targeting the same entity to detect a version mismatch. This reflects realistic intra-session behaviour when a player modifies the same entity more than once.

During development, this benchmark identified a quadratic performance issue in the initial implementation. The original `ReplayAll` called `MarkFlushed` once per resolved command, each of which performed a linear scan of the pending vector. This produced $O(n^2)$ behaviour: 50,000 commands took 2,939 ms before the fix. The fix — collecting all resolved keys and removing them in a single pass — reduced that to 9 ms, a 313-fold improvement.

6.3 Command Log Backend Comparison

The second benchmark compares the in-memory `CommandLog` against the SQLite-backed `SQLiteCommandLog` for append and retrieval operations.

Commands	Mem. Append (ms)	SQL Append (ms)	Mem. Get (ms)	SQL Get (ms)
100	0.00	0.33	0.00	0.03
1,000	0.01	3.35	0.00	0.20
5,000	0.06	19.96	0.02	1.06
10,000	0.13	33.57	0.03	1.82

SQLite append is approximately 250 times slower than in-memory (33 ms vs 0.13 ms for 10,000 commands), which is expected given that SQLite writes to disk with ACID guarantees. In absolute terms the overhead is still acceptable for an offline session: a player generating 10,000 actions — far more than a typical session — would incur roughly 33 ms of storage overhead. Retrieval via `GetPending` is fast in both backends; at 10,000 commands SQLite takes under 2 ms.

6.4 Conflict Detection Overhead

The third benchmark holds the command count fixed at 10,000 and varies the proportion of commands that are guaranteed to produce a version conflict, from 0% to 100%.

Conflict rate	Time (ms)	Throughput (cmd/s)	Applied
0%	1.10	9,130,686	3,246
25%	0.92	10,898,185	3,213
50%	0.93	10,696,617	3,175
75%	0.98	10,163,025	3,085
100%	0.89	11,230,176	2,912

Throughput varies by less than 20% across the full range of conflict rates. Higher conflict rates are marginally faster because rejected commands skip the database write step. This confirms that the version check and classification logic add negligible overhead and that the system degrades gracefully under adversarial conflict conditions.

6.5 Full Offline Session Round-Trip

The fourth benchmark measures the complete cost of an offline session using the SQLite command log: appending commands to the durable log, retrieving them, and replaying against the in-memory database.

Commands	Append (ms)	GetPending (ms)	ReplayAll (ms)	Total (ms)
100	0.30	0.02	0.15	0.47
1,000	2.86	0.17	0.81	3.84
5,000	15.26	0.84	3.09	19.19
10,000	30.42	1.70	6.29	38.40

A full session of 10,000 commands completes in approximately 38 ms end-to-end. SQLite append accounts for roughly 80% of that time; actual replay takes only 6 ms. All three phases scale linearly with command count. The results indicate that reconnection latency attributable to synchronization is unlikely to be noticeable in practice for sessions of realistic size.

6.6 Command Log Disk Footprint

The fifth benchmark measures how much disk space the SQLite-backed command log consumes as a function of log size. Commands are generated with UUID-length idempotency keys (36 characters) to reflect realistic production identifiers. File size is measured after the database connection is closed, including any WAL file that was not checkpointed.

Commands	File size (KB)	Bytes per command
100	32.0	327.7
500	80.0	163.8
1,000	140.0	143.4
5,000	628.0	128.6
10,000	1,248.0	127.8

The per-command cost stabilizes at approximately 128 bytes for logs of 1,000 commands or more. This overhead has two components: the raw data content of each row (idempotency key at 36 bytes, type string at approximately 13 bytes, entity ID, two integers, and timestamp — roughly 75 bytes in total) and SQLite's structural overhead from the B-tree page layout and the secondary index on the `idempotency_key` column, which adds approximately 50 bytes per row.

The high per-command cost at small log sizes (327 bytes at 100 commands) is a consequence of SQLite's fixed page size of 4 KB: even a single-row database occupies at least two pages — one for the data table and one for the unique index. Once the log grows large enough to fill pages efficiently the overhead drops to the steady-state figure.

In practical terms, a 1,000-command offline session — already a large session by most standards — occupies 140 KB. A session of 10,000 commands occupies approximately 1.2 MB. These figures are negligible on any modern storage medium and confirm that disk space is not a limiting factor for the command log approach.

7 Discussion

7.1 Architectural Trade-offs

Persistent state management in online multiplayer games can be implemented using a wide range of architectural strategies, each with different trade-offs between consistency, flexibility, and implementation complexity.

A purely server-authoritative architecture provides strong consistency by maintaining a single source of truth, but it cannot support offline play and becomes unavailable the moment a player loses connectivity. Snapshot-based synchronization allows entire world states to be stored locally and merged upon reconnection, but merging divergent snapshots becomes increasingly difficult as the world model grows more complex — particularly in survival games with player-owned structures, faction relations, and quest progression that interact in non-trivial ways.

The command-based approach proposed here sits between these two extremes. By recording player intent as serialized commands rather than storing divergent state snapshots, synchronization occurs through ordered replay rather than structural merging. This preserves the existing transactional guarantees of the backend while keeping the local state representation minimal. Compared to adopting CRDTs or operational transformation, it also requires substantially less redesign of the existing persistence layer, at the cost of requiring careful conflict classification and server-side validation.

7.2 Evaluation Findings

The benchmark results support the architectural claims in several concrete ways.

The replay throughput experiments show that processing a command log of 10,000 entries takes under 2 ms in the in-memory case and under 7 ms including SQLite retrieval. These figures suggest that reconnection latency attributable to the synchronization mechanism itself is unlikely to be perceptible in practice. The data transmitted on reconnection scales with the number of actions taken offline rather than with the size of the world, which is a meaningful advantage over snapshot-based approaches in large persistent worlds.

The conflict rate experiment provides a particularly useful result: throughput varies by less than 20% across the full range of conflict rates, from 0% to 100%. This means the system does not degrade meaningfully under adversarial conditions, and conflict detection can be considered a negligible overhead rather than a potential bottleneck.

The benchmarking process also revealed a design flaw in the initial implementation. The original `ReplayAll` called `MarkFlushed` individually for each resolved command, each of which performed a linear scan of the pending vector. This produced $O(n^2)$ behaviour that was not apparent from code review alone: 50,000 commands took nearly 3 seconds before the fix and 9 ms after. This illustrates

the value of performance testing as part of the design process, not just as a post-hoc validation step.

7.3 Scalability Considerations

The benchmarks characterise the performance of a single-client replay on one machine, which is a necessary but not sufficient basis for scalability claims about a large-scale multiplayer game. Several factors outside the scope of the prototype would affect performance in a production environment.

First, network latency is not modelled. Transmitting a command log over the network and waiting for server acknowledgements would dominate the reconnection time for large logs, dwarfing the sub-millisecond replay costs measured here. Second, the server-side validation logic — checking entity versions and preconditions — was evaluated against an in-memory store. Against a real SQL database under concurrent load from many reconnecting players, validation throughput would depend heavily on database indexing, connection pooling, and transaction isolation levels. Third, the benchmarks use a synthetic workload with uniformly distributed entity access. Real player behaviour may be bursty and spatially clustered, which could affect conflict rates and cache behaviour in ways not captured by the synthetic experiments.

These limitations do not undermine the core design, but they do mean that the benchmark results should be treated as indicative of single-client algorithmic performance rather than a prediction of production capacity.

7.4 Limitations and Threats to Validity

The conflict resolution strategy handles the three command classes identified in this thesis — idempotent, commutative, and non-commutative — but does not cover all edge cases that a production system would encounter. The most significant gap is commands that depend on entities which were deleted server-side during the offline session: these are caught by the existence check in `ReplayCommand`, but the player receives only a rejection notification with no automatic recovery path. A production system would need policies for how to surface and resolve these failures in a way that makes sense within the game's design.

The evaluation workload was generated synthetically using a uniform random distribution over entities and command types. It is not known whether this distribution is representative of real player behaviour, which likely varies by game genre, session length, and player experience level. Collecting telemetry from real offline sessions would allow future work to calibrate the benchmark parameters and test the system under more realistic conditions.

7.5 Game Preservation

Beyond the technical design, offline functionality has an important consequence for the long-term availability of games as cultural artefacts. Games that depend on continuous server connectivity cease to function when the developer shuts down the online service, whether due to bankruptcy, acquisition, or commercial decision. Building in offline capability means that the game remains playable independently of the publisher's infrastructure, which is directly relevant to current policy debates around game preservation (Ondruška et al., 2026).

8 Future Work

Several directions for future investigation emerge from the limitations identified in this thesis.

Telemetry-calibrated benchmarks. The evaluation used synthetically generated command logs with uniform random distributions. Collecting real telemetry from offline gameplay sessions would allow the benchmark parameters — entity access distributions, session lengths, conflict rates — to be calibrated against actual player behaviour. This would give a more reliable picture of expected performance and surface corner cases that synthetic workloads may miss.

Production server-side scaling. The server-side replay was evaluated against an in-memory entity store. A production deployment would run validation against a live SQL database under concurrent load from many clients reconnecting simultaneously. Future work should measure how the validation pipeline scales with database contention, test appropriate transaction isolation levels, and explore batching strategies for the server-side replay endpoint.

Network latency modelling. Replay throughput was measured locally, but in practice the command log is transmitted over a network before server-side validation begins. A more complete evaluation would model round-trip latency and measure how command log size affects reconnection time across different network conditions.

Recovery paths for rejected commands. The current design returns a rejection manifest to the client but does not specify what the game should do with it. A production system would need player-facing flows for communicating which offline actions were declined and offering resolution options — for example, refunding consumed resources or offering equivalent substitute items. Designing these recovery policies in a way that is both technically correct and satisfying to players is a non-trivial game-design problem.

Command dependency graphs. Some commands may depend on earlier commands in the same offline log — for example, building a structure and then upgrading it. If the first command is rejected, the second is also invalid regardless of its version check. Modelling explicit command dependencies would allow the replay engine to reject dependent chains early and produce more informative rejection manifests.

9 Conclusion

This thesis investigated the following research question: How can synchronization algorithms for persistent game state be designed and evaluated with respect to consistency, conflict resolution, and scalability in large-scale survival multiplayer games?

The starting point was a server-authoritative persistence architecture in which all state changes are executed as transactional SQL operations against a centralized cloud backend. While robust in fully online environments, this model assumes continuous connectivity and does not support offline progression or hybrid play modes. Enabling offline functionality therefore requires architectural changes that preserve data integrity while allowing temporary divergence of state.

Several architectural strategies were considered. Direct local database mirroring through SQLite provides a familiar persistence interface during offline sessions but does not itself address how divergent state is reconciled upon reconnection. An object–relational mapping layer was also evaluated but rejected due to integration complexity and the risk of disrupting the existing persistence boundary.

The proposed solution is a command-based replication model. Rather than synchronizing complete world snapshots, the system records player intent as serializable commands that can be replayed deterministically against the authoritative server. This approach addresses the three dimensions of the research question as follows.

Consistency is maintained by treating the cloud server as the single authoritative source of truth. Offline commands are not committed until they have been validated and replayed server-side. The existing transactional guarantees of the SQL backend are preserved throughout.

Conflict resolution is handled through a combination of entity versioning, command classification, and per-type resolution policies. Idempotent commands are replayed unconditionally; commutative operations such as resource deltas are merged by summation; and non-commutative operations are validated against current server state before being accepted or rejected. Commands that cannot be applied are returned to the client in a rejection manifest rather than being silently discarded.

Scalability is supported by the architecture's reliance on the existing server-authoritative infrastructure. Because synchronization occurs through ordered command replay rather than full state diffing, the volume of data transmitted on reconnection is proportional to the number of offline actions rather than the total world size.

The approach introduces its own requirements: commands must be deterministic and replay-safe, ordering must be preserved across the command log, and server-side validation logic must cover all command types to prevent exploita-

tion. These constraints are manageable within the scope of the existing architecture but would require careful engineering in a production system.

Overall, the results suggest that a command-based synchronization model provides a practical and extensible foundation for hybrid offline–online persistence in survival video games, and that consistent synchronization can be achieved without fundamentally redesigning the existing server-authoritative framework.

References

Baker, N. (2025). *Online gaming statistics*. https://www.uswitch.com/broadband/studies/online-gaming-statistics/

Bernstein, P. A., & Goodman, N. (1981). Concurrency control in distributed database systems. *ACM Computing Surveys*, *13*(2), 185–221. https://doi.org/10.1145/356842.356846

Corporation, V. (2023). *Steam cloud*. https://partner.steamgames.com/doc/features/cloud

Ellis, C. A., & Gibbs, S. J. (1989). Concurrency control in groupware systems. *ACM SIGMOD Record*, *18*(2), 399–407. https://doi.org/10.1145/66926.66963

Fowler, M. (2005). *Event sourcing*. https://martinfowler.com/eaaDev/EventSourcing.html

Hevner, A. R., March, S. T., Park, J., & Ram, S. (2004). Design science in information systems research. *MIS Quarterly*, *28*(1), 75–105. https://doi.org/10.2307/25148625

Nystrom, R. (2014). *Game programming patterns*. Genever — Benning. https://books.google.no/books?id=9fIwBQAAQBAJ

Ondruška, D., Scott, R., Katzner, M.-M., Cerezo, A. H., & Barczyk, M. (2026). *Stop killing games at the european parliament*. https://www.youtube.com/watch?v=QXdmoeaYZ9Y

Shapiro, M., Preguiça, N., Baquero, C., & Zawirski, M. (2011). Conflict-free replicated data types, 386–400. https://doi.org/10.1007/978-3-642-24550-3_29

Studio, I. G. (2021). *Valheim*. https://www.valheimgame.com